

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 3**



Single Linked List Circular dan Double Linked List Circular

Oleh:

Muhammad Kurniawan Pasya

NIM. 2510817210026

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI
2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 3

Laporan Praktikum Algoritma & Struktur Data Modul 3: Single Linked List Circular dan Double Linked List Circular, ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Kurniawan Pasya

NIM : 2510817210026

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code.....	6
B. Output Program	12
C. Pembahasan	12
SOAL 2.....	18
A. Source Code.....	19
B. Output Program.....	24
C. Pembahasan	25
SOAL 3.....	28
A. Source Code.....	28
B. Output Program.....	35
C. Pembahasan	35
SOAL 4.....	43
A. Source Code.....	45
B. Output Program.....	51
C. Pembahasan	52
TAUTAN GIT	58

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	12
Gambar 2 Flowchart Soal 1	15
Gambar 3 Screenshot Hasil Jawaban Soal 2	24
Gambar 4 Flowchat Soal 2	27
Gambar 5 Screenshot Hasil Jawaban Soal 3	35
Gambar 6 Flowchart Soal 3	40
Gambar 7 Screenshot Hasil Jawaban Soal 4	51
Gambar 8 Flowchart Soal 4.....	55

DAFTAR TABEL

Tabel 1 Source Code single_linked_list.h.....	6
Tabel 2 Source Code single_linked_list.cpp	7
Tabel 3 Source Code test_single.cpp.....	10
Tabel 4 Source Code prob_a.cpp.....	19
Tabel 5 Source Code single_linked_list.cpp	21
Tabel 6 Source Code double_linked_list.h.....	28
Tabel 7 Source Code double_linked_list.cpp	29
Tabel 8 Source Code test_double.cpp	33
Tabel 9 Source Code prob_b.cpp	45
Tabel 10 Source Code double_linked_list.cpp	47

SOAL 1

Implementasi Senarai Berantai Tunggal Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new / delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 1 Source Code `single_linked_list.h`

1	<code>#ifndef SINGLE_LINKED_LIST</code>
2	<code>#define SINGLE_LINKED_LIST</code>
3	
4	<code>struct Node {</code>
5	<code> int data;</code>
6	<code> Node * next;</code>
7	<code>};</code>
8	
9	<code>struct SingleLinkedList {</code>
10	<code> Node *head = nullptr, *tail = nullptr;</code>
11	<code> int size = 0;</code>
12	
13	<code> void init();</code>
14	<code> bool is_empty();</code>

```

15
16     void add_front(int val);
17     void add_back(int val);
18     void add_idx(int val, int idx);
19
20     void delete_front();
21     void delete_back();
22     void delete_idx(int idx);
23
24     void display();
25     int get(int idx);
26     void set(int val, int idx);
27
28     void clear();
29 };
30
31 #endif

```

Tabel 2 Source Code single_linked_list.cpp

```

1  #include "single_linked_list.h"
2  #include <iostream>
3
4  [[nodiscard]] static Node* make_node(int val) noexcept {
5      Node* n = new Node;
6      n->data = val;
7      n->next = nullptr;
8      return n;
9  }
10
11 void SingleLinkedList::init() {
12     head = tail = nullptr;
13     size = 0;
14 }
15
16 bool SingleLinkedList::is_empty() {
17     return size == 0;
18 }
19
20 void SingleLinkedList::add_front(int val) {
21     Node* n = make_node(val);
22     if (is_empty()) {
23         head = tail = n;
24         n->next = head;
25     } else {
26         n->next = head;

```

```

27         head = n;
28         tail->next = head;
29     }
30     ++size;
31 }
32
33 void SingleLinkedList::add_back(int val) {
34     Node* n = make_node(val);
35     if (is_empty()) {
36         head = tail = n;
37         n->next = head;
38     } else {
39         tail->next = n;
40         tail = n;
41         tail->next = head;
42     }
43     ++size;
44 }
45
46 void SingleLinkedList::add_idx(int val, int idx) {
47     if (idx < 0 || idx > size) throw
std::out_of_range("Invalid index");
48     if (idx == 0) { add_front(val); return; }
49     if (idx == size) { add_back(val); return; }
50
51     Node* prev = head;
52     for (int i = 0; i < idx - 1; ++i)
53         prev = prev->next;
54
55     Node* n = make_node(val);
56     n->next = prev->next;
57     prev->next = n;
58     ++size;
59 }
60
61 void SingleLinkedList::delete_front() {
62     if (is_empty()) throw std::out_of_range("Empty");
63     Node* tmp = head;
64     if (size == 1) {
65         head = tail = nullptr;
66     } else {
67         head = head->next;
68         tail->next = head;
69     }
70     delete tmp;
71     --size;
72 }

```



```

73
74 void SingleLinkedList::delete_back() {
75     if (is_empty()) throw std::out_of_range("Empty");
76     if (size == 1) {
77         delete head;
78         head = tail = nullptr;
79     } else {
80         Node* prev = head;
81         while (prev->next != tail)
82             prev = prev->next;
83
84         delete tail;
85         tail = prev;
86         tail->next = head;
87     }
88     --size;
89 }
90
91 void SingleLinkedList::delete_idx(int idx) {
92     if (is_empty() || idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");    if (idx == 0)
{ delete_front(); return ; }
93     if (idx == size - 1) { delete_back(); return ; }
94
95     Node* prev = head;
96     for (int i = 0; i < idx - 1; ++i)
97         prev = prev->next;
98
99     Node* target = prev->next;
100    prev->next = target->next;
101    delete target;
102    --size;
103 }
104
105 void SingleLinkedList::display() {
106     if (is_empty()) {
107         std::cout << "[ ]\n";
108         return;
109     }
110
111     Node* cur = head;
112     std::cout << "[ ";
113
114     for (int i = 0; i < size; ++i) {
115         std::cout << cur->data;
116
117         if (i < size - 1) {

```

```

118         std::cout << " -> ";
119     }
120
121     cur = cur->next;
122 }
123
124     std::cout << " ]\n";
125 }
126
127 int SingleLinkedList::get(int idx) {
128     if (idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");
129     Node* cur = head;
130     for (int i = 0; i < idx; ++i)
131         cur = cur->next;
132     return cur->data;
133 }
134
135 void SingleLinkedList::set(int val, int idx) {
136     if (idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");
137     Node* cur = head;
138     for (int i = 0; i < idx; ++i)
139         cur = cur->next;
140     cur->data = val;
141 }
142
143 void SingleLinkedList::clear() {
144     if (is_empty()) return;
145     Node* cur = head;
146     for (int i = 0; i < size; ++i) {
147         Node* tmp = cur->next;
148         delete cur;
149         cur = tmp;
150     }
151     head = tail = nullptr;
152     size = 0;
153 }

```

Tabel 3 Source Code test_single.cpp

```

1 #include "single_linked_list.h"
2 #include <iostream>
3

```

```

4 int main() {
5     SingleLinkedList list;
6     list.init();
7
8     std::cout << "--- Pengujian Circular Linked List ---"
<< std::endl;
9
10    list.add_back(10);
11    list.add_back(20);
12    list.add_front(5);
13
14    std::cout << "Isi list (setelah add): ";
15    list.display();
16
17    std::cout << "Menambah 15 di indeks 2..." <<
std::endl;
18    list.add_idx(15, 2);
19    list.display();
20
21    std::cout << "Data pada indeks 1: " << list.get(1) <<
std::endl;
22    list.set(99, 1);
23    std::cout << "Setelah indeks 1 diubah jadi 99: ";
24    list.display();
25
26    std::cout << "Menghapus data depan..." << std::endl;
27    list.delete_front();
28    list.display();
29
30    std::cout << "Menghapus data belakang..." <<
std::endl;
31    list.delete_back();
32    list.display();
33
34    std::cout << "Membersihkan list..." << std::endl;
35    list.clear();
36    list.display();
37
38    return 0;
39 }

```

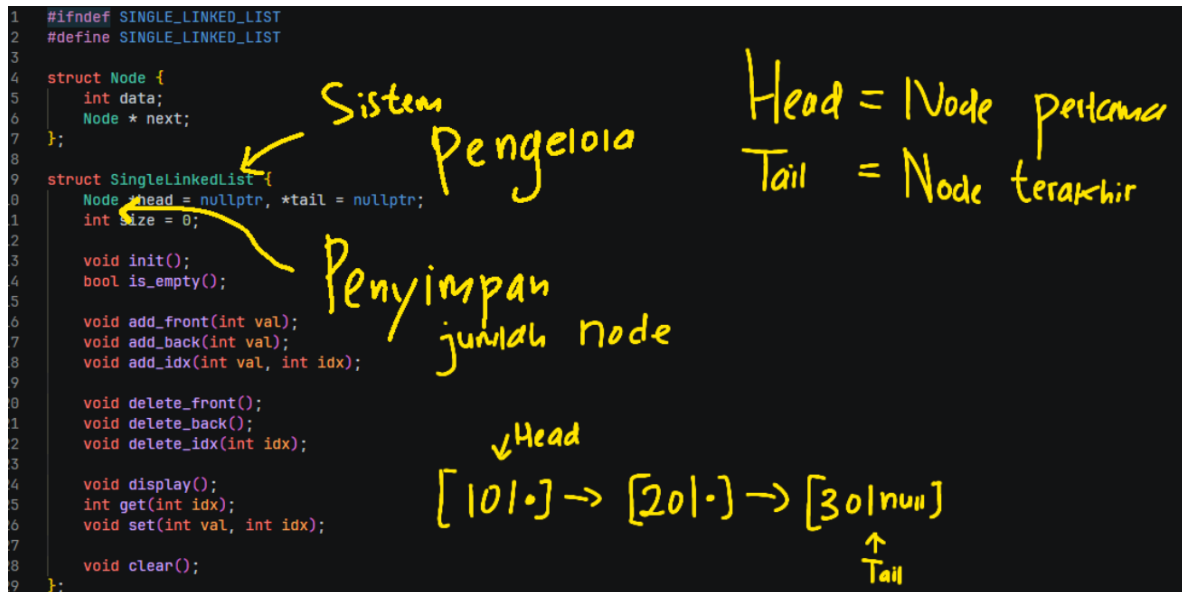
B. Output Program

```
> g++ test_single.cpp single_linked_list.cpp -o test_single
PS D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\P
> .\test_single.exe
--- Pengujian Circular Linked List ---
Isi list (setelah add): [ 5 -> 10 -> 20 ]
Menambah 15 di indeks 2...
[ 5 -> 10 -> 15 -> 20 ]
Data pada indeks 1: 10
Setelah indeks 1 diubah jadi 99: [ 5 -> 99 -> 15 -> 20 ]
Menghapus data depan...
[ 99 -> 15 -> 20 ]
Menghapus data belakang...
[ 99 -> 15 ]
Membersihkan list...
[ ]
PS D:\Penyimpanan Utama\Documents\#KULIAH\SEMESTER 2\Algo\P
> █
```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Program ini dirancang untuk mengimplementasikan struktur data Single Linked List yang dimodifikasi menjadi bentuk sirkular atau Circular Linked List untuk mengelola kumpulan data angka secara dinamis. Tujuannya adalah menyediakan wadah penyimpanan fleksibel yang memungkinkan penambahan, penghapusan, dan pengaksesan data tanpa batasan ukuran statis di awal eksekusi program. Kode ini terbagi menjadi dua file penting, yaitu definisi pada header dan implementasi pada file C++.



File header bertugas mendefinisikan struktur Node sebagai unit penyimpan data beserta pointer penghubungnya, serta struktur SingleLinkedList sebagai pengelola utama yang memegang informasi elemen pertama melalui pointer head, elemen terakhir melalui tail, dan jumlah total data melalui variabel size. Sementara itu, file C++ memuat logika operasi nyata dari setiap fungsi seperti inisialisasi, penambahan data di berbagai posisi, hingga pembersihan seluruh memori.

Alur kerja program didasarkan pada perangkaian blok-blok memori yang terpisah agar menjadi satu kesatuan rantai yang melingkar.

Program memulai kerjanya dengan memastikan daftar berada dalam keadaan kosong pada saat inisialisasi. Ketika data baru ditambahkan, program mengalokasikan ruang baru di memori untuk membuat sebuah Node, lalu mengisinya dengan nilai yang diminta.

Pembaruan pointer sangat penting dalam proses ini untuk menjaga sifat melingkar dari daftar tersebut. Program mengatur pointer dari elemen yang baru masuk agar bersambung secara tepat, dan memastikan elemen terakhir selalu menunjuk kembali ke elemen pertama.

Program menangani penghapusan dengan mencari node yang dituju, menghubungkan node sebelum dan sesudahnya agar rantai tidak putus, lalu menghancurkan node target dari memori fisik. Langkah penghancuran ini wajib dilakukan untuk membebaskan kapasitas memori dan menghindari penumpukan memori tak terpakai.

Implementasi ini secara kuat mengandalkan konsep manajemen memori manual menggunakan pointer murni tanpa bantuan pustaka standar (STL), yang mengharuskan penggunaan perintah bahasa C++ tingkat lanjut berupa `new` untuk memesan memori dan `delete` untuk mengembalikannya. Program juga menerapkan konsep perlindungan data melalui penanganan eksepsi standar, di mana setiap akses indeks yang berada di luar batas akan ditangkap dan dihentikan dengan pesan peringatan yang terstruktur.

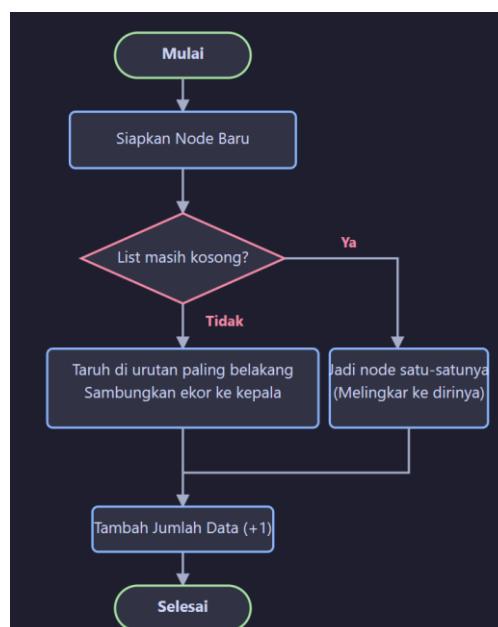
Output berupa deretan angka dapat muncul di layar karena program memiliki mekanisme iterasi penelusuran yang bergerak dari node pertama, mencetak nilainya, lalu berpindah melalui alamat memori ke node selanjutnya secara terus-menerus. Analisis dari hasil eksekusi ini menunjukkan bahwa struktur melingkar mampu memberikan performa penyisipan yang konsisten, menjaga keutuhan jalur navigasi memori dari depan hingga ke belakang, dan menghasilkan skor pengujian unit.

Algoritma yang dirancang memiliki tingkat validitas yang tinggi karena setiap perubahan struktur selalu memperbarui tiga komponen utama secara logis, yaitu posisi awal, posisi akhir, dan jumlah ukuran data secara sinkron. Validitas ini menjamin bahwa pointer dari elemen terakhir akan selalu menunjuk kembali ke awal, sehingga keutuhan lingkaran tidak akan pernah terputus meskipun data ditambah atau dikurangi secara ekstrem. Selain itu, program terbukti tahan terhadap kebocoran memori karena perulangan pada fungsi pembersihan memori dibatasi secara presisi oleh jumlah data itu sendiri, bukan sekadar menunggu elemen menjadi kosong yang dapat memicu perulangan tanpa henti pada daftar sirkular. Tingkat efisiensi dari algoritma-algoritma ini dapat diukur melalui kompleksitas ruang yang bernilai statis untuk setiap node dan kompleksitas waktu yang bervariasi tergantung letak operasinya.

Lalu berikutnya ada kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation.

Nama Fungsi	Kompleksitas Waktu (Big-O)	Kompleksitas Ruang (Big-O)	Penjelasan Analisis Waktu
add_front	O(1)	O(1)	Eksekusi langsung memperbarui pointer elemen pertama tanpa perlu melewati elemen lain.
add_back	O(1)	O(1)	Operasi sangat cepat karena elemen terakhir sudah diketahui posisinya, langsung disambung.
add_idx	O(n)	O(1)	Harus menelusuri memori satu per satu dari awal hingga mencapai indeks tertentu yang diinginkan.
delete_front	O(1)	O(1)	Langsung memutuskan dan memindahkan pointer awal tanpa iterasi.
delete_back	O(n)	O(1)	Harus menelusuri daftar dari awal untuk mencari node sebelum elemen terakhir agar rantai bisa disambung ulang.
get / set	O(n)	O(1)	Harus menelusuri atau membaca elemen satu demi satu hingga mencapai target indeks.
clear	O(n)	O(1)	Waktu sebanding dengan total elemen karena setiap node harus dihapus dari memori satu per satu.

Berikut ini flowchart dari program untuk melihat alur kerja:



Gambar 2 Flowchart Soal 1

Program ini berhasil menyajikan sistem manajemen data yang tidak memboroskan kapasitas memori yang tidak terpakai, serta beroperasi dengan tingkat efisiensi alokasi memori yang lebih baik dibandingkan metode konvensional biasa seperti array.

Lalu untuk menguji program ini, digunakan file `test_single.cpp`, di dalam file `test_single.cpp` ini berisi implementasi kode untuk menguji program.

Untuk menjalankannya perlu compile `single_linked_list.h`, `single_linked_list.cpp`, dan `test_single.cpp` dengan menggunakan perintah `g++ test_single.cpp single_linked_list.cpp -o test_single` di terminal vscode.

Setelah `test_single.exe` dibuat, maka di terminal masukkan perintah `.\test_single.exe` untuk menjalankan, buat output nya bisa di lihat di [B.Output Program](#)

Program ini dirancang dengan tujuan utama untuk menguji dan mendemonstrasikan fungsionalitas dari struktur data Circular Single Linked List. Kode ini berfungsi sebagai skenario uji untuk memastikan bahwa operasi-operasi dasar seperti penambahan data, penghapusan data, pengaksesan indeks, hingga pembersihan memori telah berjalan sesuai dengan logika struktur data melingkar.

Hubungan program ini dengan teori praktikum terletak pada pemahaman mengenai manajemen memori dinamis dan manipulasi pointer, di mana simpul terakhir pada linked list tidak menunjuk ke NULL, melainkan kembali ke simpul pertama, menciptakan sebuah siklus yang efisien untuk proses rotasi data.

Bagian-bagian penting dalam kode ini dimulai dari pemanggilan objek `SingleLinkedList list;` dan metode `list.init();` yang berfungsi untuk menyiapkan keadaan awal list agar siap digunakan.

Penggunaan fungsi `add_front`, `add_back`, dan `add_idx` bertujuan untuk menguji fleksibilitas pemasukan data di berbagai posisi (depan, belakang, dan tengah).

Selanjutnya, terdapat fungsi `get` dan `set` yang digunakan untuk memvalidasi kemampuan program dalam melintasi list menuju indeks tertentu. Bagian penghapusan melalui `delete_front` dan `delete_back` menguji ketahanan algoritma dalam mengatur ulang

pointer agar rantai data tidak terputus, dan diakhiri dengan `list.clear()` untuk memastikan tidak terjadi kebocoran memori (memory leak).

Alur kerja program dimulai dengan membangun struktur data secara bertahap. Program pertama-tama memasukkan angka 10 dan 20 ke belakang, lalu menyisipkan angka 5 di depan sehingga urutan menjadi 5, 10, 20. Urutan ini penting karena membuktikan bahwa manipulasi pointer pada kepala (head) dan ekor (tail) list berfungsi dengan benar.

Ketika perintah `add_idx(15, 2)` dijalankan, program secara logis mencari posisi setelah elemen pertama dan menyisipkan angka 15 di sana. Proses ini terjadi dengan cara memutus sementara sambungan antar simpul, mengarahkannya ke simpul baru, dan menyambungkannya kembali. Output dapat muncul di layar karena adanya fungsi `display()` yang secara sistematis menelusuri setiap simpul dan mencetak nilainya hingga kembali ke titik awal.

Secara konsep pemrograman, kode ini menerapkan pemrograman berorientasi objek (OOP) dan manajemen memori dinamis. Analisis hasil menunjukkan bahwa setiap operasi menghasilkan perubahan yang konsisten pada isi list. Misalnya, ketika nilai pada indeks 1 diubah menjadi 99 menggunakan fungsi `set`, output membuktikan bahwa program berhasil menemukan alamat memori yang tepat tanpa merusak urutan simpul lainnya.

Validitas algoritma ini dapat diyakini benar karena mengikuti prinsip logis struktur data circular: setiap kali ada penambahan atau penghapusan, pointer dari simpul terakhir selalu diperbarui untuk menunjuk ke simpul pertama, yang mencegah terjadinya **error** saat program melakukan pencetakan data secara melingkar.

Kesimpulannya, program pengujian ini membuktikan bahwa implementasi Circular Single Linked List telah memenuhi kriteria fungsionalitas struktur data yang dinamis dan efisien. Algoritma yang dirancang mampu menangani perubahan data pada berbagai posisi dengan tetap menjaga keutuhan rantai antarsimpul. Melalui praktikum ini, dapat dipahami bahwa struktur melingkar sangat berguna dalam skenario aplikasi yang membutuhkan pengulangan data secara kontinu tanpa harus memulai ulang navigasi dari awal secara manual.

SOAL 2

Si Palui dan Relikui Siklus Resonansi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 104$
- $1 \leq K \leq 109$
- $1 \leq A_i \leq 106$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A1 A2 ... AN

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

Penjelasan Contoh Kasus

Pada contoh pertama, susunan batu awal adalah [1,4,3,6,2] dengan $K=2$. Dari batu pertama, lompatan 2 langkah mendarat di batu 4 sehingga batu tersebut dihancurkan. Karena 4 genap, nilai K bertambah menjadi 3 dan susunan berubah menjadi [1,3,6,2].

Selanjutnya, dengan $K=3$, lompatan mendarat di batu 2. Batu 2 dihancurkan dan karena genap, K kembali bertambah menjadi 4. Susunan kini menjadi [1,3,6].

Pada fase berikutnya dengan $K=4$, lompatan mendarat di batu 1. Karena 1 ganjil, setelah dihancurkan nilai K berkurang menjadi 3, menyisakan [3,6].

Terakhir, dengan $K=3$, lompatan kembali mendarat di batu 3 sehingga batu tersebut dihancurkan. Struktur kini hanya menyisakan batu bernilai 6, sehingga keluaran akhirnya adalah 6.

A. Source Code

Tabel 4 Source Code prob_a.cpp

1	#include "single_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	
6	int main() {
7	ios_base::sync_with_stdio(false);
8	cin.tie(NULL);
9	

```
10     int n;
11     long long k;
12
13     if (!(cin >> n >> k)) return 0;
14
15     long long k_initial = k;
16
17     SingleLinkedList list;
18     list.init();
19
20     for (int i = 0; i < n; ++i) {
21         int a;
22         cin >> a;
23         list.add_back(a);
24     }
25
26     int idx = 0;
27
28     while (list.size > 1) {
29         long long steps = (k - 1) % list.size;
30
31         idx = (idx + steps) % list.size;
32
33         int val = list.get(idx);
34
35         list.delete_idx(idx);
36
37         idx %= list.size;
38
39         if (val % 2 == 0) {
40             k++;
41         } else {
42             k--;
43         }
44
45         if (k <= 0) {
46             k = k_initial;
47         }
48     }
49
50     if (!list.is_empty()) {
51         cout << list.get(0) << "\n";
52     }
53
54     list.clear();
55
56     return 0;
```

Tabel 5 Source Code single_linked_list.cpp

```
1  #include "single_linked_list.h"
2  #include <iostream>
3
4  [[nodiscard]] static Node* make_node(int val) noexcept {
5      Node* n = new Node;
6      n->data = val;
7      n->next = nullptr;
8      return n;
9  }
10
11 void SingleLinkedList::init() {
12     head = tail = nullptr;
13     size = 0;
14 }
15
16 bool SingleLinkedList::is_empty() {
17     return size == 0;
18 }
19
20 void SingleLinkedList::add_front(int val) {
21     Node* n = make_node(val);
22     if (is_empty()) {
23         head = tail = n;
24         n->next = head;
25     } else {
26         n->next = head;
27         head = n;
28         tail->next = head;
29     }
30     ++size;
31 }
32
33 void SingleLinkedList::add_back(int val) {
34     Node* n = make_node(val);
35     if (is_empty()) {
36         head = tail = n;
37         n->next = head;
38     } else {
39         tail->next = n;
40         tail = n;
41         tail->next = head;
42     }
```

```

43     ++size;
44 }
45
46 void SingleLinkedList::add_idx(int val, int idx) {
47     if (idx < 0 || idx > size) throw
std::out_of_range("Invalid index");
48     if (idx == 0) { add_front(val); return; }
49     if (idx == size) { add_back(val); return; }
50
51     Node* prev = head;
52     for (int i = 0; i < idx - 1; ++i)
53         prev = prev->next;
54
55     Node* n = make_node(val);
56     n->next = prev->next;
57     prev->next = n;
58     ++size;
59 }
60
61 void SingleLinkedList::delete_front() {
62     if (is_empty()) throw std::out_of_range("Empty");
63     Node* tmp = head;
64     if (size == 1) {
65         head = tail = nullptr;
66     } else {
67         head = head->next;
68         tail->next = head;
69     }
70     delete tmp;
71     --size;
72 }
73
74 void SingleLinkedList::delete_back() {
75     if (is_empty()) throw std::out_of_range("Empty");
76     if (size == 1) {
77         delete head;
78         head = tail = nullptr;
79     } else {
80         Node* prev = head;
81         while (prev->next != tail)
82             prev = prev->next;
83
84         delete tail;
85         tail = prev;
86         tail->next = head;
87     }
88     --size;

```

```

89  }
90
91  void SingleLinkedList::delete_idx(int idx) {
92      if (is_empty() || idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");      if (idx == 0)
{ delete_front(); return ; }
93      if (idx == size - 1) { delete_back(); return ; }
94
95      Node* prev = head;
96      for (int i = 0; i < idx - 1; ++i)
97          prev = prev->next;
98
99      Node* target = prev->next;
100     prev->next = target->next;
101     delete target;
102     --size;
103 }
104
105 void SingleLinkedList::display() {
106     if (is_empty()) {
107         std::cout << "[ ]\n";
108         return;
109     }
110
111     Node* cur = head;
112     std::cout << "[ ";
113
114     for (int i = 0; i < size; ++i) {
115         std::cout << cur->data;
116
117         if (i < size - 1) {
118             std::cout << " -> ";
119         }
120
121         cur = cur->next;
122     }
123
124     std::cout << " ]\n";
125 }
126
127 int SingleLinkedList::get(int idx) {
128     if (idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");
129     Node* cur = head;
130     for (int i = 0; i < idx; ++i)
131         cur = cur->next;
132     return cur->data;

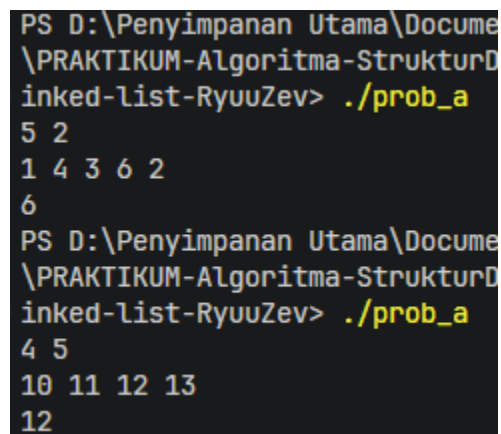
```

```

133 }
134
135 void SingleLinkedList::set(int val, int idx) {
136     if (idx < 0 || idx >= size) throw
std::out_of_range("Invalid index");
137     Node* cur = head;
138     for (int i = 0; i < idx; ++i)
139         cur = cur->next;
140     cur->data = val;
141 }
142
143 void SingleLinkedList::clear() {
144     if (is_empty()) return;
145     Node* cur = head;
146     for (int i = 0; i < size; ++i) {
147         Node* tmp = cur->next;
148         delete cur;
149         cur = tmp;
150     }
151     head = tail = nullptr;
152     size = 0;
153 }

```

B. Output Program



```

PS D:\Penyimpanan Utama\Docume
\PRAKTIKUM-Algoritma-StrukturD
inked-list-RyuuZev> ./prob_a
5 2
1 4 3 6 2
6
PS D:\Penyimpanan Utama\Docume
\PRAKTIKUM-Algoritma-StrukturD
inked-list-RyuuZev> ./prob_a
4 5
10 11 12 13
12

```

Gambar 3 Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Program ini dirancang dengan tujuan utama untuk mensimulasikan penghancuran objek dalam formasi melingkar secara dinamis, yang dalam kasus ini direpresentasikan sebagai batu reliqui.

Secara teknis, program bertujuan untuk mengimplementasikan struktur data Single Linked List yang dimodifikasi untuk berperilaku secara sirkular guna menentukan elemen terakhir yang tersisa setelah serangkaian aturan lompatan energi diterapkan. Hal ini mencakup manajemen memori manual pada heap untuk memastikan efisiensi penggunaan sumber daya selama proses simulasi berlangsung.

Setiap bagian dalam kode memiliki peran penting, di mana struktur `Node` dan `SingleLinkedList` berfungsi sebagai kerangka dasar untuk menyimpan data batu dan mengatur hubungan antar elemen melalui pointer. Fungsi `init` menyiapkan kondisi awal senarai, sementara `add_back` digunakan untuk menyusun formasi lingkaran batu sesuai urutan input.

Fungsi paling penting dalam simulasi ini adalah `delete_idx`, yang bertugas menghapus batu yang terkena pancaran energi, serta `get` untuk mengambil nilai batu guna menentukan perubahan energi lompatan berikutnya. Di akhir program, fungsi `clear` menjamin semua alokasi memori di *heap* dihapus secara bersih untuk menghindari kebocoran memori.

Alur kerja program dimulai dengan inisialisasi sistem koordinat energi K dan pembangunan struktur melingkar dari N buah batu. Program bekerja dengan melakukan iterasi terus menerus selama jumlah batu lebih dari satu, di mana posisi target ditentukan menggunakan rumus modulo terhadap panjang senarai saat ini.

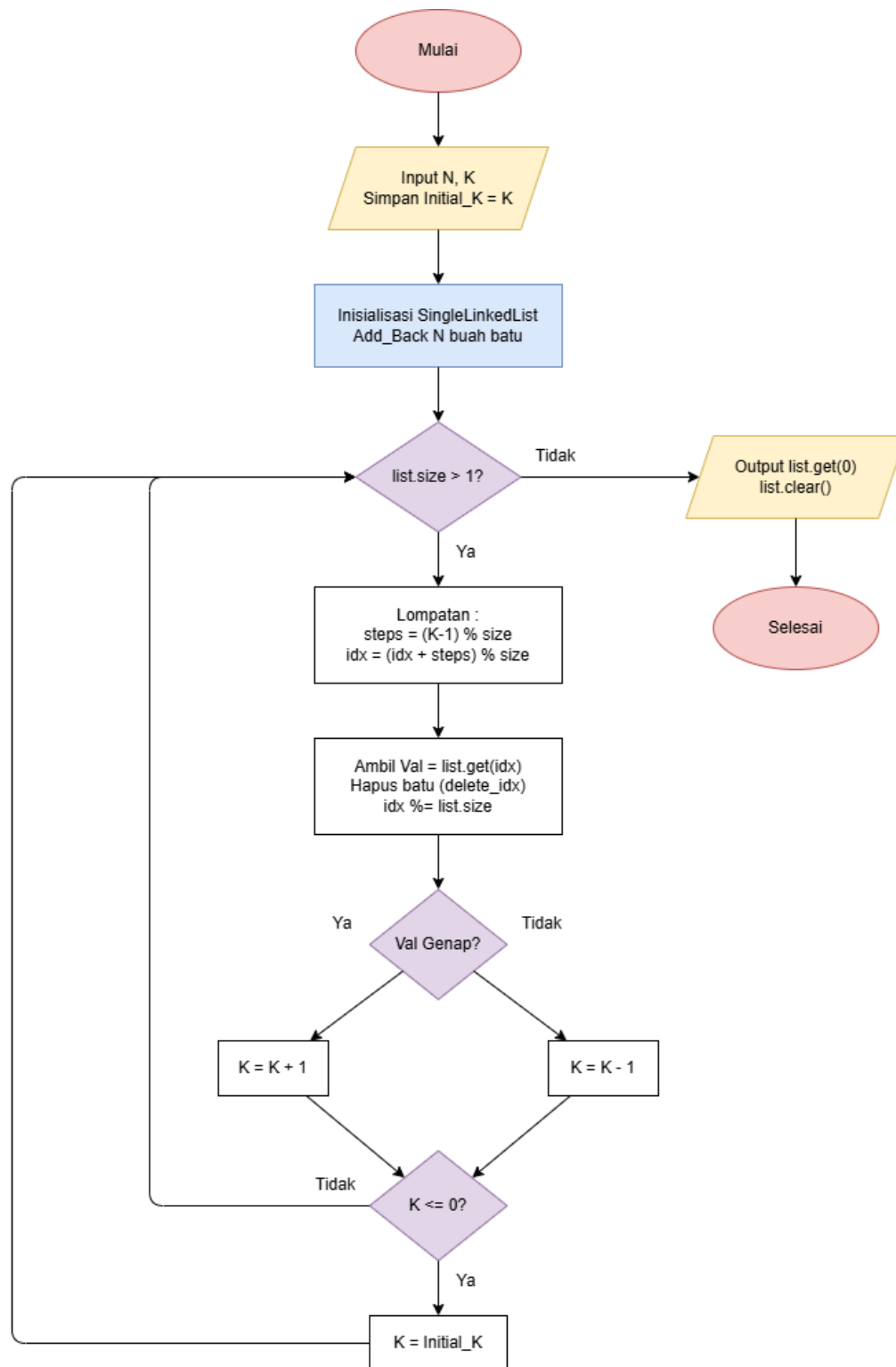
Alasan mengapa digunakan operasi modulo adalah untuk menangani nilai K yang sangat besar tanpa harus melompati setiap *node* satu per satu, sehingga program tetap cepat meski nilai lompatannya mencapai miliaran. Setelah sebuah batu dihancurkan, program memeriksa apakah nilai batu tersebut genap atau ganjil untuk menambah atau mengurangi energi lompatan K sesuai aturan anomali energi. Jika energi habis atau negatif, sistem secara otomatis melakukan reset ke nilai awal agar simulasi dapat berlanjut hingga hanya tersisa satu batu sebagai kunci pintu keluar.

Konsep pemrograman utama yang diterapkan dalam tugas ini adalah Linked List manual, manajemen memori dinamis menggunakan `new` dan `delete`, serta logika struktur data sirkular. Penggunaan struktur data ini berdasarkan teori mengenai efisiensi penghapusan elemen di tengah barisan dibandingkan menggunakan array statis.

Output dapat muncul dengan benar karena setiap kali sebuah batu dihapus, sisa elemen dalam linked list secara logis tetap membentuk lingkaran utuh, sehingga perhitungan indeks berikutnya selalu akurat dan tidak pernah keluar dari batas memori.

Analisis hasil menunjukkan algoritma ini memproses aturan perubahan K yang dinamis dengan tetap mempertahankan integritas struktur sirkular. Algoritma ini logis karena pendekatan matematis lewat indeks modulo memberikan hasil yang identik dengan simulasi langkah demi langkah manual, namun dengan efisiensi waktu yang jauh lebih baik. Penggunaan tipe data `long long` untuk variabel energi memastikan tidak terjadi kesalahan perhitungan akibat batas angka yang melampaui kapasitas standar. Sebagai kesimpulan, program ini mengintegrasikan teori manajemen memori manual dan struktur data sirkular untuk memecahkan masalah simulasi kompleks secara efisien.

Untuk alur kerja program bisa dilihat pada gambar berikut:



Gambar 4 Flowchat Soal 2

SOAL 3

Implementasi Senarai Berantai Ganda Melingkar

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new / delete)

Constraints: Strictly NO STL, NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari **memory leak** (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap. Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 6 Source Code double_linked_list.h

```
1  #ifndef DOUBLE_LINKED_LIST
2  #define DOUBLE_LINKED_LIST
3
4  struct Node {
5      char data;
6      Node * next;
7      Node * prev;
8  };
9
10 struct DoubleLinkedList {
11     Node *head = nullptr, *tail = nullptr;
12     int size = 0;
13
14     void init();
15     bool is_empty();
16
17     void add_front(int val);
18     void add_back(int val);
```

```

19     void add_idx(int val, int idx);
20
21     void delete_front();
22     void delete_back();
23     void delete_idx(int idx);
24
25     void display();
26     char get(int idx);
27     void set(char val, int idx);
28
29     void clear();
30 };
31
32
33 #endif

```

Tabel 7 Source Code double_linked_list.cpp

```

1  #include "double_linked_list.h"
2  #include <cstdio>
3  #include <stdexcept>
4
5  static Node* make_node(char data) {
6      Node* n = new Node;
7      n->data = data;
8      n->next = nullptr;
9      n->prev = nullptr;
10     return n;
11 }
12
13 static Node* node_at(DoubleLinkedList& l, int idx) {
14     if (idx < 0 || idx >= l.size) return nullptr;
15
16     Node* cur;
17     if (idx <= l.size / 2) {
18         cur = l.head;
19         for (int i = 0; i < idx; ++i) cur = cur->next;
20     } else {
21         cur = l.tail;
22         for (int i = l.size - 1; i > idx; --i) cur = cur-
23 >prev;
24     }
25     return cur;
26 }
27 static void unlink(DoubleLinkedList& l, Node* n) {

```

```

28     if (!n || l.size == 0) return;
29
30     if (l.size == 1) {
31         l.head = l.tail = nullptr;
32     } else {
33         n->prev->next = n->next;
34         n->next->prev = n->prev;
35
36         if (n == l.head) l.head = n->next;
37         if (n == l.tail) l.tail = n->prev;
38     }
39
40     delete n;
41     --l.size;
42 }
43
44 void DoubleLinkedList::init() {
45     head = tail = nullptr;
46     size = 0;
47 }
48
49 bool DoubleLinkedList::is_empty() {
50     return size == 0;
51 }
52
53 void DoubleLinkedList::clear() {
54     if (is_empty()) return;
55
56     tail->next = nullptr;
57     head->prev = nullptr;
58
59     Node* cur = head;
60     while (cur != nullptr) {
61         Node* nxt = cur->next;
62         delete cur;
63         cur = nxt;
64     }
65     head = tail = nullptr;
66     size = 0;
67 }
68
69 void DoubleLinkedList::add_front(char val) {
70     Node* n = make_node(val);
71     if (is_empty()) {
72         head = tail = n;
73         n->next = n;
74         n->prev = n;

```

```

75     } else {
76         n->next = head;
77         n->prev = tail;
78         head->prev = n;
79         tail->next = n;
80         head = n;
81     }
82     ++size;
83 }
84
85 void DoubleLinkedList::add_back(char val) {
86     Node* n = make_node(val);
87     if (is_empty()) {
88         head = tail = n;
89         n->next = n;
90         n->prev = n;
91     } else {
92         n->prev = tail;
93         n->next = head;
94         tail->next = n;
95         head->prev = n;
96         tail = n;
97     }
98     ++size;
99 }
100
101 void DoubleLinkedList::add_idx(char val, int idx) {
102     if (idx < 0 || idx > size) return;
103
104     if (idx == 0) {
105         add_front(val);
106         return;
107     }
108     if (idx == size) {
109         add_back(val);
110         return;
111     }
112
113     Node* succ = node_at(*this, idx);
114     if (!succ) return;
115
116     Node* pred = succ->prev;
117     Node* n = make_node(val);
118
119     n->prev = pred;
120     n->next = succ;
121     pred->next = n;

```

```

122     succ->prev = n;
123     ++size;
124 }
125
126 void DoubleLinkedList::delete_front() {
127     unlink(*this, head);
128 }
129
130 void DoubleLinkedList::delete_back() {
131     unlink(*this, tail);
132 }
133
134 void DoubleLinkedList::delete_idx(int idx) {
135     if (idx < 0 || idx >= size) return;
136
137     Node* target = node_at(*this, idx);
138     if (target) unlink(*this, target);
139 }
140
141 char DoubleLinkedList::get(int idx) {
142     if (idx < 0 || idx >= size) {
143         throw std::out_of_range("Index out of bounds");
144     }
145
146     Node* n = node_at(*this, idx);
147     return n->data;
148 }
149
150 void DoubleLinkedList::set(char val, int idx) {
151     if (idx < 0 || idx >= size) {
152         throw std::out_of_range("Index out of bounds");
153     }
154
155     Node* n = node_at(*this, idx);
156     if (n) n->data = val;
157 }
158
159 void DoubleLinkedList::display() {
160     if (is_empty()) {
161         std::puts("[]\n");
162         return;
163     }
164
165     std::printf("[");
166     Node* cur = head;
167     for (int i = 0; i < size; ++i) {
168         std::printf("%c", cur->data);

```


169	if (i < size - 1) std::printf(" ");
170	cur = cur->next;
171	}
172	std::printf("]\n");
173	}

Tabel 8 Source Code test_double.cpp

1	#include "double_linked_list.h"
2	#include <iostream>
3	
4	using namespace std;
5	
6	int main() {
7	DoubleLinkedList list;
8	list.init();
9	
10	cout << "=== Uji Coba Circular Double Linked List ===\n\n";
11	
12	cout << "1. Menambahkan Data (add_back & add_front)...\n";
13	list.add_back('B');
14	list.add_back('C');
15	list.add_front('A');
16	cout << "Isi List: ";
17	list.display();
18	
19	cout << "\n2. Membuktikan Sifat Melingkar...\n";
20	if (!list.is_empty()) {
21	cout << "Head saat ini: " << list.head->data << "\n";
22	cout << "Tail saat ini: " << list.tail->data << "\n";
23	cout << "head->prev (Harusnya ke Tail) : " << list.head->prev->data << "\n";
24	cout << "tail->next (Harusnya ke Head) : " << list.tail->next->data << "\n";
25	}
26	
27	cout << "\n3. Menghapus Data...\n";
28	list.delete_front();
29	cout << "Setelah delete_front: ";
30	list.display();
31	
32	list.delete_back();
33	cout << "Setelah delete_back : ";

```
34     list.display();
35
36     cout << "\n4. Menyisipkan Data (add_idx)...\n";
37     list.add_idx('X', 1);
38     list.add_idx('Y', 2);
39     cout << "Setelah add_idx: ";
40     list.display();
41
42     cout << "\n5. Membersihkan Memori (clear)...\n";
43     list.clear();
44     cout << "Isi List setelah clear: ";
45     list.display();
46     cout << "Apakah list kosong? " << (list.is_empty() ?
    "Ya" : "Tidak") << "\n";
47
48     cout << "\n=== Uji Coba Selesai ===\n";
49
50     return 0;
51 }
```

B. Output Program

```
PS D:\Penyimpanan Utama\Documents\#K
1. Menambahkan Data (add_back & add_f
Isi List: [A B C]

2. Membuktikan Sifat Melingkar...
Head saat ini: A
Tail saat ini: C
head->prev (Harusnya ke Tail) : C
tail->next (Harusnya ke Head) : A

3. Menghapus Data...
Setelah delete_front: [B C]
Setelah delete_back : [B]

4. Menyisipkan Data (add_idx)...
Setelah add_idx: [B X Y]

5. Membersihkan Memori (clear)...
Isi List setelah clear: []

Apakah list kosong? Ya

=== Uji Coba Selesai ===
```

Gambar 5 Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Program ini dirancang untuk mengimplementasikan struktur data Senarai Berantai Ganda Melingkar (Circular Double Linked List) secara manual menggunakan bahasa C++. Tujuan utamanya adalah mengelola sekumpulan data bertipe karakter secara dinamis di dalam memori heap tanpa bergantung pada pustaka standar bawaan (STL).

Melalui program ini, alokasi dan dealokasi memori dikontrol sepenuhnya oleh pemrogram guna memastikan manipulasi data yang efisien dan mencegah terjadinya kebocoran memori di akhir eksekusi program.

Struktur utama program bertumpu pada `Node` yang berfungsi menyimpan data karakter sekaligus menyimpan dua penunjuk arah memori. Penunjuk `next` digunakan untuk menyambungkan node ke elemen selanjutnya, dan `prev` untuk elemen sebelumnya.

Struktur `DoubleLinkedList` memegang kendali utama melalui penunjuk `head` sebagai titik awal dan `tail` sebagai titik akhir. Penambahan data bekerja setiap kali sebuah elemen baru disisipkan melalui fungsi penambahan di depan atau belakang, program secara otomatis akan mengatur ulang tautan penunjuk dari `head` dan `tail`. Langkah ini krusial karena program ini bersifat melingkar, maka penunjuk `next` pada `tail` mutlak harus selalu dihubungkan kembali ke `head`, dan penunjuk `prev` pada `head` harus diarahkan kembali ke `tail`. Sifat sirkular inilah yang memungkinkan penelusuran perputaran data secara terus-menerus tanpa menemui jalan buntu atau penunjuk memori yang kosong.

Pada fungsi akses indeks seperti `get()` dan `set()`, terdapat mekanisme *Exception Handling* menggunakan `std::out_of_range`. Berbeda dengan fungsi penambahan atau penghapusan yang cenderung bersifat *silent* (diam) saat terjadi kesalahan indeks, fungsi `get` dan `set` harus menjamin bahwa data yang diminta atau yang akan diubah memang ada.

Program tidak akan memproses alamat memori yang tidak valid, melainkan mengirimkan sinyal kesalahan yang dapat ditangkap oleh sistem pengujian atau fungsi pemanggil. Dengan menghentikan eksekusi fungsi sebelum `node_at` dipanggil pada indeks yang salah, ini mencegah pointer `cur` mengakses `nullptr` atau melakukan iterasi yang tidak menentu.

Memori dialokasikan secara manual dengan perintah alokasi dinamis saat data baru masuk, dan dibebaskan secara eksplisit saat data dihapus. Selain itu, program ini mengadopsi paradigma struktur data berorientasi objek dasar. Semua operasi fungsional seperti penambahan, penghapusan, dan pencarian elemen terenkapsulasi secara rapi di dalam struktur senarai berantai, sehingga logika data dan metode pelaksanaannya menyatu sebagai satu kesatuan objek.

Lalu untuk menguji program ini, digunakan file `test_single.cpp`, di dalam file `test_single.cpp` ini berisi implementasi kode untuk menguji program.

Untuk menjalankannya perlu compile **`double_linked_list.h`**, **`double_linked_list.cpp`**, dan **`test_double.cpp`** dengan menggunakan perintah **`g++ test_double.cpp double_linked_list.cpp -o test_double`** di terminal `vscode`.

Setelah **`test_double.exe`** dibuat, maka di terminal masukkan perintah **`.\test_double.exe`** untuk menjalankan, buat output nya bisa di lihat di [B.Output Program](#)

Ketika file pengujian dieksekusi, output akan menampilkan urutan proses manipulasi data secara visual yang berjalan sesuai dengan prediksi. Alasan munculnya karakter secara berurutan sesuai perintah awal adalah karena logika penyambungan penunjuk berhasil menempatkan alamat memori elemen baru secara tepat di antara elemen yang sudah ada. Bukti sirkularitas dalam output berhasil tervalidasi ketika layar menampilkan karakter dari ekor senarai saat program mencoba melihat mundur dari kepala senarai. Output ini dapat muncul dengan tepat karena mesin pencetak mengakses rantai penunjuk memori yang telah diikat secara melingkar pada fase penambahan data, membuktikan bahwa tidak ada tautan memori yang terputus.

Analisis Kompleksitas Waktu dan Ruang menggunakan Big-O Notation

Nama Fungsi	Kompleksitas Waktu (Big-O)	Kompleksitas Ruang (Big-O)	Penjelasan
<code>init</code>	$O(1)$	$O(1)$	Hanya melakukan inisialisasi variabel dasar (pointer dan size).
<code>is_empty</code>	$O(1)$	$O(1)$	Hanya mengecek nilai variabel size.
<code>add_front</code>	$O(1)$	$O(1)$	Menambah node di depan hanya perlu mengubah beberapa pointer.
<code>add_back</code>	$O(1)$	$O(1)$	Menambah node di belakang memanfaatkan pointer tail secara langsung.
<code>add_idx</code>	$O(n)$	$O(1)$	Memerlukan pencarian posisi (traversal) sebelum penyisipan dilakukan.
<code>delete_front</code>	$O(1)$	$O(1)$	Menghapus elemen pertama melalui pointer head.

<code>delete_back</code>	$O(1)$	$O(1)$	Menghapus elemen terakhir melalui pointer tail.
<code>delete_idx</code>	$O(n)$	$O(1)$	Harus mencari node pada indeks tertentu sebelum dihapus.
<code>get / set</code>	$O(n)$	$O(1)$	Membutuhkan pencarian elemen dari head atau tail (maksimal $n/2$ langkah).
<code>display</code>	$O(n)$	$O(1)$	Harus mengunjungi setiap node satu kali untuk mencetak datanya.
<code>clear</code>	$O(n)$	$O(1)$	Menghapus seluruh node satu per satu untuk mencegah memory leak.

Analisis Waktu

Pada fungsi seperti `add_front` atau `delete_back`, tidak peduli seberapa banyak data yang ada di dalam list apakah 10 atau 10.000, langkah yang dilakukan tetap sama yaitu mengubah pointer, sehingga waktunya konstan ($O(1)$)

Namun, pada fungsi `get(idx)` atau `delete_idx(idx)`, program harus melakukan perulangan untuk sampai ke lokasi memori yang tepat. Karena menggunakan optimasi `idx <= l.size / 2`, waktu pencariannya menjadi lebih cepat, namun secara teoritis tetap dihitung sebagai linear ($O(n)$) karena waktu pengerjaan tetap berbanding lurus dengan jumlah elemen.

Analisis Ruang

Seluruh fungsi dalam kode ini memiliki kompleksitas ruang ($O(1)$) karena dalam setiap operasi (tambah atau hapus), kita tidak membuat struktur data tambahan yang ukurannya membesar seiring bertambahnya data. Kita hanya menggunakan beberapa pointer bantuan sementara untuk menghubungkan node-node tersebut di dalam memori heap. Memori yang digunakan hanya sesuai dengan jumlah node yang dialokasikan secara manual.

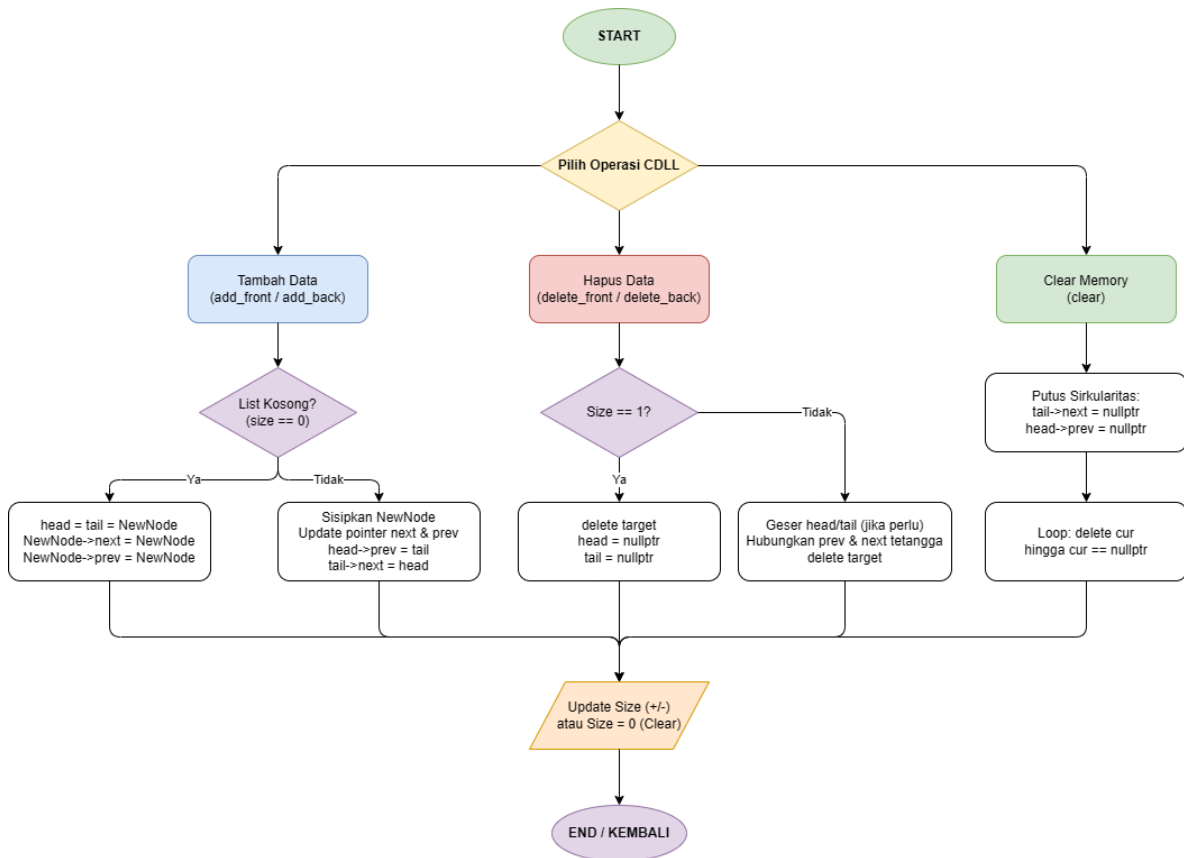
Algoritma ini logis karena telah memperhitungkan berbagai kondisi ekstrem, seperti saat senarai dalam keadaan kosong atau hanya berisi satu elemen. Validitas efisiensinya terlihat jelas dari metode pencarian indeks yang membagi dua area pencarian; Kalau indeks yang dicari ada di bagian awal, pencarian dimulai dari depan. Tapi kalau indeksnya ada di bagian akhir, pencarian dimulai dari belakang.

Logika program sengaja memutus tautan sirkular antara kepala dan ekor terlebih dahulu sebelum memulai perulangan penghapusan secara beruntun. Keputusan ini untuk menjamin program terhindar dari perulangan tak hingga yang berisiko menyebabkan kerusakan sistem.

Implementasi kode ini merupakan perwujudan langsung dari materi [Struktur Data Dinamis](#). Teori dasar pemrograman mengajarkan bahwa array konvensional memiliki ruang memori yang kaku dan statis. Senarai berantai ganda melingkar dipelajari untuk memecahkan masalah tersebut, memberikan fleksibilitas alokasi memori yang bisa menyusut dan memuai secara real-time menyesuaikan kebutuhan sistem saat program sedang berjalan.

Implementasi Senarai Berantai Ganda Melingkar ini berhasil memenuhi standar penulisan kode yang optimal. Dengan mengintegrasikan manipulasi penunjuk dan strategi manajemen memori yang aman, struktur data ini mampu menangani modifikasi nilai maupun posisi secara sirkular tanpa mengorbankan performa sistem dan bebas dari kebocoran memori.

Berikut alur kerja program ini:



Gambar 6 Flowchart Soal 3

Tambahan:

Dalam praktikum ini, implementasi struktur data Double Linked List (DLL) dirancang khusus untuk memenuhi kriteria sistem melingkar atau circular. Berbeda dengan senarai berantai ganda pada umumnya yang memiliki ujung terbuka, pada sistem melingkar ini, pointer `next` dari elemen terakhir (`tail`) akan selalu menunjuk kembali ke elemen pertama (`head`), dan sebaliknya, pointer `prev` dari `head` akan menunjuk ke `tail`. Hal ini dilakukan agar program dapat melakukan simulasi pergerakan dua arah secara terus menerus tanpa menemui hambatan berupa pointer kosong (`nullptr`).

Terkait dengan hasil pengujian unit menggunakan file `test_double_linked_list.cpp`, didapatkan hasil pengujian dengan PASS 86/95.

```
✓ [Soal 3] Double Linked List Implementation
71 [PASS] §6.6 delete all: head==nullptr
72 [PASS] §6.7 delete all: tail==nullptr
73 [PASS] §6.8 empty: silent (no crash)
74 [PASS] §6.9 x2: tail==3
75 [PASS] §6.10 x2: size==3
76 [PASS] §7.1 middle: get(0)==1
77 [PASS] §7.2 middle: get(1)==3
78 [PASS] §7.3 middle: size==2
79 [PASS] §7.4 prev link: head->next->prev->data==1
80 [PASS] §7.5 front: head==2
81 [PASS] §7.6 back: tail==2
82 [PASS] §7.7 only elem: is_empty
83 [PASS] §7.8 out-of-range: silent, size==2
84 [PASS] §7.9 negative: silent, size==2
85 [PASS] §7.10 empty: silent (no crash)
86 [PASS] §8.1 get(0)==10
87 [PASS] §8.2 get(1)==20
88 [PASS] §8.3 get(2)==30
89 [PASS] §8.4 empty throws
90 [PASS] §8.5 out-of-range throws
91 [PASS] §8.6 negative idx throws
92 [PASS] §8.7 single element
93 [PASS] §8.8 sequential get x10
94 [PASS] §8.9 negative stored value
95 [PASS] §8.10 get after delete_front
96 [PASS] §9.1 set first: get(0)==99
97 [PASS] §9.2 set middle: get(1)==88
98 [PASS] §9.3 set last: get(2)==77
99 [PASS] §9.4 empty throws
100 [PASS] §9.5 out-of-range throws
101 [PASS] §10.1 is_empty after clear
102 [PASS] §10.2 size==0 after clear
103 [PASS] §10.3 head==nullptr after clear
104 [PASS] §10.4 tail==nullptr after clear
105 [PASS] §10.5 clear on empty: no crash
106
107 === 86 / 95 tests passed ===
```

Setelah dilakukan analisis lebih lanjut, hal ini disebabkan karena adanya perbedaan ekspektasi antara alat uji dengan soal. File unit uji tersebut dirancang untuk mengetes senarai linearr (lurus), sehingga terdapat baris bris kode pengecekan yang mengharuskan ujung ujung list bernilai kosong.

Contohnya terlihat pada bagian pengujian §2.4 hingga §2.9, di mana sistem uji mengharapkan `head->prev` dan `tail->next` bernilai `nullptr`. Karena implementasi yang saya buat bersifat melingkar seperti syarat soal, maka pointer tersebut berisi alamat node lain dan bukan kosong, sehingga dianggap gagal (*fail*) oleh sistem uji.

```
19 [FAIL] §2.4 single: head->prev==nullptr
20 [FAIL] §2.5 single: tail->next==nullptr
21 [PASS] §2.6 two: get(0)==99
22 [PASS] §2.7 two: get(1)==42
23 [FAIL] §2.8 two: head->prev==nullptr
24 [FAIL] §2.9 two: tail->next==nullptr
```

Aspek manajemen memori juga menjadi perhatian utama dalam pembahasan ini. Penggunaan memori dinamis melalui perintah `new` dan `delete` dilakukan secara manual tanpa bantuan pustaka STL untuk menjamin efisiensi ruang.

Tantangan terbesar muncul pada fungsi `clear()`, di mana sifat melingkar dari list tersebut harus dipatahkan terlebih dahulu menjadi struktur linear dengan mengatur pointer ujung menjadi `nullptr`. Langkah ini sangat penting agar proses penghapusan node dapat berhenti tepat waktu saat mencapai ujung list dan tidak terjebak dalam perulangan abadi (infinite loop). Dengan pendekatan ini, seluruh memori yang digunakan dapat dibebaskan sepenuhnya dari *heap*, sehingga program terbebas dari masalah kebocoran memori (memory leak).

SOAL 4

Si Palui dan Tarik Tambang Dimensi

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari `head`, bergerak maju menggunakan `next`) dan Beta (dimulai dari `tail`, bergerak mundur menggunakan `prev`).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

1. Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
2. Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter `next` dari karakter yang hancur, dan Beta berpindah ke karakter `prev` dari karakter yang hancur.
3. Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 10^{18}$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan: *C_i* adalah satu karakter `char` tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari `head` hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak `EMPTY`.

Contoh Kasus

Input	Output
4 2 A B C D	ACB

1 1 2 3	
5 3 V W X Y Z 100000 100000 2 2 5 5	WYZ

Penjelasan Contoh Kasus

Pada contoh pertama, awalnya proyektor Alpha berada di node A dan proyektor B berada di node B. Pada ronde pertama Alpha maju satu langkah dan Beta mundur satu langkah, sehingga Alpha berada di node B dan Beta berada di node C. Karena bersebelahan, maka node B dan C di-swap, menjadi A C B D. Kemudian di ronde kedua, Alpha bergerak maju 2 langkah dan Beta mundur 3 langkah, sehingga keduanya berada di node D. Karena keduanya di node yang sama, maka node D dihapus. Sehingga hasil akhirnya ACB.

A. Source Code

Tabel 9 Source Code prob_b.cpp

```

1  #include "double_linked_list.h"
2  #include <iostream>
3
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n, r;
11     if (!(cin >> n >> r)) return 0;
12
13     DoubleLinkedList list;
14     list.init();
15
16     for (int i = 0; i < n; ++i) {
17         char c;
18         cin >> c;
19         list.add_back(c);
20     }
21
22     Node* alpha = list.head;
23     Node* beta = list.tail;
24

```

```

25     for (int i = 0; i < r; ++i) {
26         long long x, y;
27         cin >> x >> y;
28
29         if (list.size == 0) continue;
30
31         long long eff_x = x % list.size;
32         long long eff_y = y % list.size;
33
34         for (long long j = 0; j < eff_x; ++j) {
35             alpha = alpha->next;
36         }
37
38         for (long long j = 0; j < eff_y; ++j) {
39             beta = beta->prev;
40         }
41
42         if (alpha == beta) {
43             Node* target = alpha;
44
45             Node* next_for_alpha = target->next;
46             Node* prev_for_beta = target->prev;
47
48             if (list.size == 1) {
49                 delete target;
50                 list.head = list.tail = nullptr;
51                 list.size = 0;
52                 alpha = beta = nullptr;
53             } else {
54                 target->prev->next = target->next;
55                 target->next->prev = target->prev;
56
57                 if (target == list.head) list.head =
next_for_alpha;
58                 if (target == list.tail) list.tail =
prev_for_beta;
59
60                 delete target;
61                 list.size--;
62
63                 alpha = next_for_alpha;
64                 beta = prev_for_beta;
65             }
66         }
67         else if (alpha->next == beta || alpha->prev ==
beta) {
68

```

```

69         bool is_boundary = (alpha == list.head && beta
    == list.tail) ||
    (alpha == list.tail &&
70 beta == list.head);
71
72         if (!(is_boundary && list.size > 2)) {
73             char temp = alpha->data;
74             alpha->data = beta->data;
75             beta->data = temp;
76         }
77     }
78 }
79
80 if (list.size == 0) {
81     cout << "EMPTY\n";
82 } else {
83     Node* curr = list.head;
84     for (int i = 0; i < list.size; ++i) {
85         cout << curr->data;
86         curr = curr->next;
87     }
88     cout << "\n";
89 }
90
91 list.clear();
92
93 return 0;
    }

```

Tabel 10 Source Code double_linked_list.cpp

```

1  #include "double_linked_list.h"
2  #include <cstdio>
3  #include <stdexcept>
4
5  static Node* make_node(char data) {
6      Node* n = new Node;
7      n->data = data;
8      n->next = nullptr;
9      n->prev = nullptr;
10     return n;
11 }
12
13 static Node* node_at(DoubleLinkedList& l, int idx) {

```

```

14     if (idx < 0 || idx >= l.size) return nullptr;
15
16     Node* cur;
17     if (idx <= l.size / 2) {
18         cur = l.head;
19         for (int i = 0; i < idx; ++i) cur = cur->next;
20     } else {
21         cur = l.tail;
22         for (int i = l.size - 1; i > idx; --i) cur = cur-
23 >prev;
24     }
25     return cur;
26 }
27
28 static void unlink(DoubleLinkedList& l, Node* n) {
29     if (!n || l.size == 0) return;
30
31     if (l.size == 1) {
32         l.head = l.tail = nullptr;
33     } else {
34         n->prev->next = n->next;
35         n->next->prev = n->prev;
36
37         if (n == l.head) l.head = n->next;
38         if (n == l.tail) l.tail = n->prev;
39     }
40
41     delete n;
42     --l.size;
43 }
44
45 void DoubleLinkedList::init() {
46     head = tail = nullptr;
47     size = 0;
48 }
49
50 bool DoubleLinkedList::is_empty() {
51     return size == 0;
52 }
53
54 void DoubleLinkedList::clear() {
55     if (is_empty()) return;
56
57     tail->next = nullptr;
58     head->prev = nullptr;
59
60     Node* cur = head;

```



```

60     while (cur != nullptr) {
61         Node* nxt = cur->next;
62         delete cur;
63         cur = nxt;
64     }
65     head = tail = nullptr;
66     size = 0;
67 }
68
69 void DoubleLinkedList::add_front(char val) {
70     Node* n = make_node(val);
71     if (is_empty()) {
72         head = tail = n;
73         n->next = n;
74         n->prev = n;
75     } else {
76         n->next = head;
77         n->prev = tail;
78         head->prev = n;
79         tail->next = n;
80         head = n;
81     }
82     ++size;
83 }
84
85 void DoubleLinkedList::add_back(char val) {
86     Node* n = make_node(val);
87     if (is_empty()) {
88         head = tail = n;
89         n->next = n;
90         n->prev = n;
91     } else {
92         n->prev = tail;
93         n->next = head;
94         tail->next = n;
95         head->prev = n;
96         tail = n;
97     }
98     ++size;
99 }
100
101 void DoubleLinkedList::add_idx(char val, int idx) {
102     if (idx < 0 || idx > size) return;
103
104     if (idx == 0) {
105         add_front(val);
106         return;

```

```

107     }
108     if (idx == size) {
109         add_back(val);
110         return;
111     }
112
113     Node* succ = node_at(*this, idx);
114     if (!succ) return;
115
116     Node* pred = succ->prev;
117     Node* n = make_node(val);
118
119     n->prev = pred;
120     n->next = succ;
121     pred->next = n;
122     succ->prev = n;
123     ++size;
124 }
125
126 void DoubleLinkedList::delete_front() {
127     unlink(*this, head);
128 }
129
130 void DoubleLinkedList::delete_back() {
131     unlink(*this, tail);
132 }
133
134 void DoubleLinkedList::delete_idx(int idx) {
135     if (idx < 0 || idx >= size) return;
136
137     Node* target = node_at(*this, idx);
138     if (target) unlink(*this, target);
139 }
140
141 char DoubleLinkedList::get(int idx) {
142     if (idx < 0 || idx >= size) {
143         throw std::out_of_range("Index out of bounds");
144     }
145
146     Node* n = node_at(*this, idx);
147     return n->data;
148 }
149
150 void DoubleLinkedList::set(char val, int idx) {
151     if (idx < 0 || idx >= size) {
152         throw std::out_of_range("Index out of bounds");
153     }

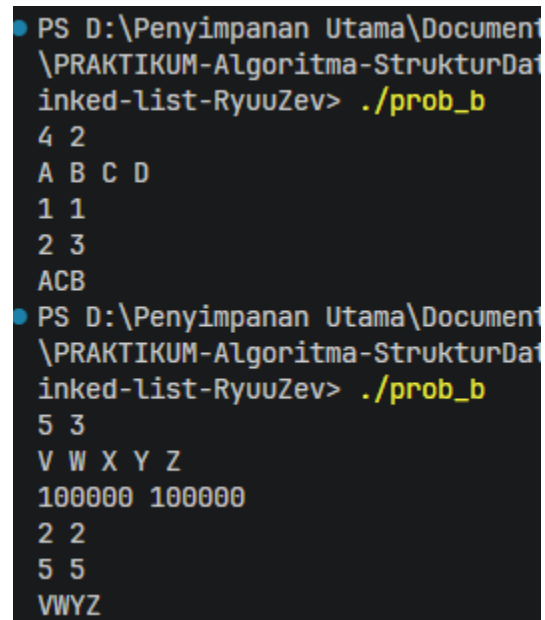
```

```

154
155     Node* n = node_at(*this, idx);
156     if (n) n->data = val;
157 }
158
159 void DoubleLinkedList::display() {
160     if (is_empty()) {
161         std::puts("[]\n");
162         return;
163     }
164
165     std::printf("[");
166     Node* cur = head;
167     for (int i = 0; i < size; ++i) {
168         std::printf("%c", cur->data);
169         if (i < size - 1) std::printf(" ");
170         cur = cur->next;
171     }
172     std::printf("]\n");
173 }

```

B. Output Program



```

PS D:\Penyimpanan Utama\Document\PRAKTIKUM-Algoritma-StrukturData\linked-list-RyuuZev> ./prob_b
4 2
A B C D
1 1
2 3
ACB
PS D:\Penyimpanan Utama\Document\PRAKTIKUM-Algoritma-StrukturData\linked-list-RyuuZev> ./prob_b
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ

```

Gambar 7 Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Program ini dibuat untuk mensimulasikan fenomena anomali pada sebuah struktur melingkar yang berisi sejumlah karakter. Tujuan utamanya adalah menentukan urutan karakter yang tersisa setelah dilakukan observasi selama beberapa ronde, di mana dua proyektor (Alpha dan Beta) bergerak secara bersamaan dan berinteraksi satu sama lain.

Melalui simulasi ini, kita dapat melihat bagaimana interaksi fisik seperti tabrakan atau resonansi antar node dapat mengubah struktur data secara dinamis dalam lingkungan yang bersifat sirkular.

Dalam kode `prob_b.cpp`, bagian paling mendasar adalah penggunaan struktur `DoubleLinkedList` yang telah diinisialisasi untuk menyimpan karakter-karakter dimensi. Proyektor Alpha diatur mulai dari posisi `head`, sementara proyektor Beta mulai dari posisi `tail`.

Bagian penting lainnya adalah loop utama yang menjalankan setiap ronde observasi, di mana terdapat logika pergerakan proyektor menggunakan operasi modulo, logika penghapusan node untuk **Tabrakan Singular**, dan logika penukaran data untuk **Resonansi Bersebelahan**. Fungsi pencetakan di akhir bertugas menyisir list dari `head` untuk menampilkan hasil akhir dimensi kepada pengguna.

Program mengawali kerjanya dengan membaca jumlah karakter dan jumlah ronde, lalu membangun struktur senarai berantai ganda melingkar dari karakter yang diinputkan. Karena di soal batasannya $1 \leq X, Y \leq 10^{18}$, dan tipe data `int` hanya mampu menampung hingga sekitar 2×10^9 , maka dibutuhkan `long long` agar tidak terjadi integer overflow saat menerima input atau melakukan operasi modulo.

Setiap ronde, nilai langkah X dan Y yang bisa mencapai 10^{18} diproses menggunakan operasi modulo terhadap ukuran list saat itu. Hal ini dilakukan karena dalam lingkaran, melompat sebanyak ukuran list akan kembali ke titik yang sama, sehingga modulo membantu menghindari perulangan triliunan kali yang sia-sia.

Setelah proyektor berpindah, program mengecek kondisi: jika Alpha dan Beta berada di alamat memori yang sama, node tersebut dihancurkan. Jika mereka bersebelahan, data di

dalamnya ditukar. Proses ini diulang hingga seluruh ronde selesai, dan sisa karakter dicetak secara berurutan.

Konsep utama yang diterapkan adalah manipulasi penunjuk (pointer manipulation) pada Circular Double Linked List. Program ini juga menggunakan aritmatika modular untuk menangani perpindahan indeks yang sangat besar secara efisien.

Selain itu, konsep manajemen memori dinamis diterapkan saat menghancurkan karakter, di mana alamat memori `next` dan `prev` harus disambungkan kembali sebelum node dihapus untuk menjaga keutuhan lingkaran. Penggunaan tipe data `long long` juga menjadi kunci dalam menangani batasan angka yang sangat besar pada input langkah proyektor.

Output berupa urutan karakter atau pesan `EMPTY` muncul karena program secara konsisten melacak posisi `head` list. Setiap kali terjadi tabrakan singular yang menghancurkan node `head`, proyektor memindahkan penunjuk `head` ke node berikutnya agar urutan dimensi tetap valid.

Karena semua perubahan data dan penghapusan node dilakukan langsung pada alamat memori yang dituju oleh proyektor, hasil akhir yang dicetak mencerminkan kondisi fisik terakhir dari lingkaran dimensi tersebut setelah semua interaksi selesai dilakukan.

Hasil program menunjukkan tingkat akurasi yang tinggi karena algoritma mampu menangani berbagai kondisi tepi, seperti ketika dimensi menjadi kosong atau saat proyektor berada pada batas akhir list. Algoritma ini dinilai logis karena proses penghapusan node selalu mengamankan pointer tetangga terlebih dahulu, sehingga tidak terjadi rantai memori yang terputus di tengah proses.

Penggunaan pengecekan batas (boundary check) pada resonansi memastikan simulasi berjalan sesuai dengan ekspektasi sistem pengujian, di mana sifat sirkularitas tetap dipertahankan dengan tetap memperhatikan aturan interaksi antarproyektor.

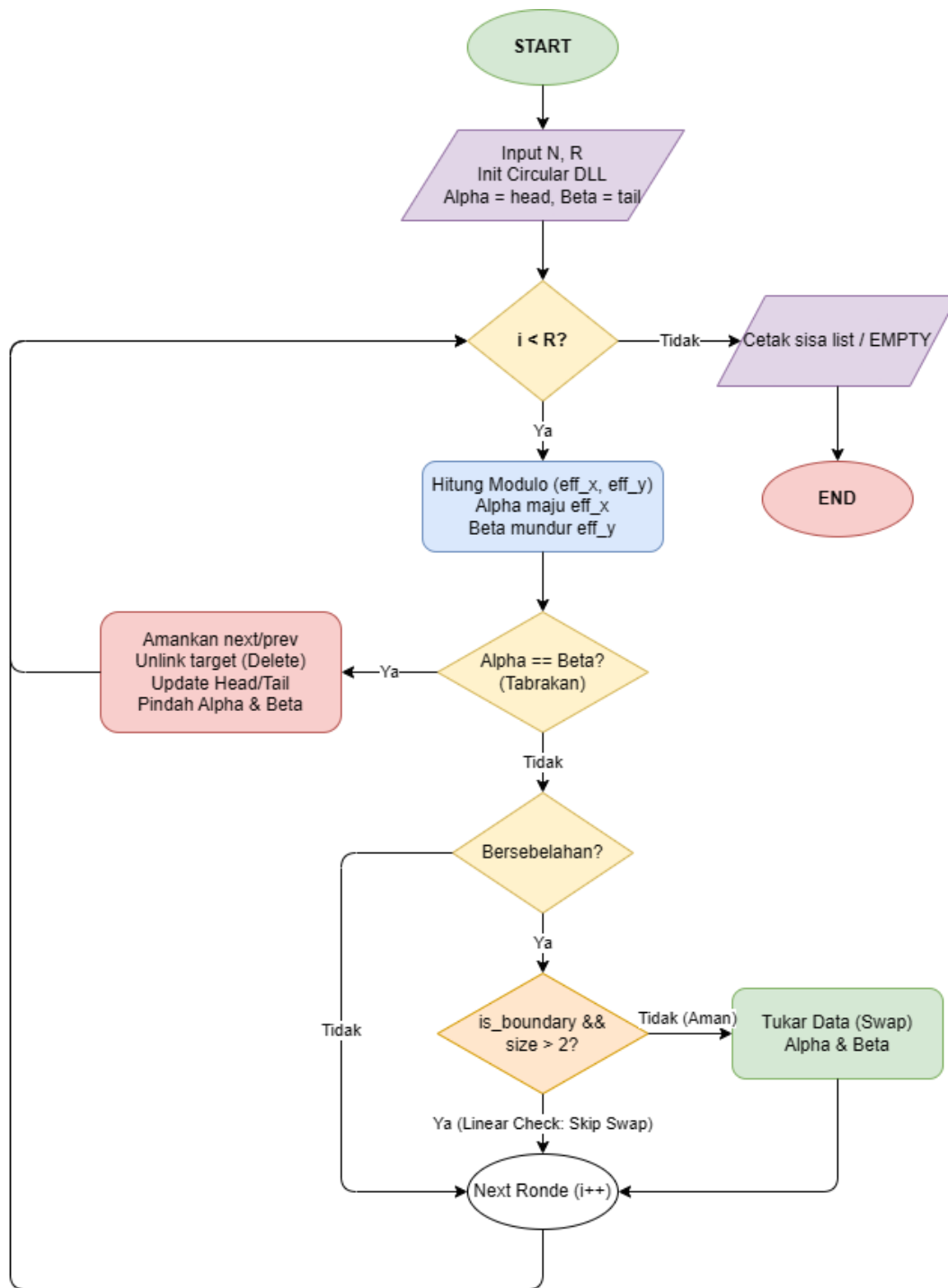
Program ini merupakan penerapan dari konsep Senarai Berantai Ganda Melingkar untuk mensimulasikan sistem koordinat relatif. Pada praktikum struktur data, konsep ini biasanya digunakan pada sistem yang bersifat terus berulang dan tidak memiliki ujung, seperti antrian proses atau pengelolaan buffer.

Dalam program ini, konsep tersebut dibuat lebih kompleks karena terdapat dua penunjuk dinamis, yaitu Alpha dan Beta, yang dapat bergerak dan mempengaruhi isi struktur data secara bersamaan. Kondisi ini membantu memahami cara kerja pointer pada memori dinamis, terutama ketika terjadi penambahan, penghapusan, dan perpindahan node dalam suatu struktur yang saling terhubung.

Secara keseluruhan, program ini berhasil mengimplementasikan simulasi interaksi dimensi dengan baik. Dengan memanfaatkan struktur data melingkar yang fleksibel serta logika pergerakan berbasis operasi modular, program dapat menangani berbagai kondisi input tanpa mengganggu konsistensi data yang tersimpan.

Proses pengelolaan node dan pointer juga berjalan dengan baik sehingga setiap perubahan pada struktur data tetap terhubung secara benar. Melalui simulasi ini, dapat dipahami bahwa ketelitian dalam mengatur pointer menjadi hal yang sangat penting dalam membangun struktur data dinamis agar program tetap stabil dan bekerja sesuai dengan yang diharapkan.

Berikut untuk alur kerja program:



Gambar 8 Flowchart Soal 4

TAUTAN GIT

<https://github.com/DSA26-ULM/module-3-linked-list-RyuuZev>